Assignment 2 – Question 3: Device Management

1. Introduction

Device management is an important function of an operating system. It allows applications to interact with hardware devices through system interfaces and device drivers.

This assignment demonstrates device management using a Python program on Parrot OS. The program interacts with storage devices by displaying available devices and retrieving information about storage capacity.

The implementation uses Linux system interfaces and Python's built-in modules to retrieve information without modifying the storage devices.

2. Objectives

The objectives of this implementation are to:

- Display available storage devices.
- Retrieve storage capacity information.
- Display total, used and available storage space.
- Implement error handling for invalid paths and inaccessible devices.
- Demonstrate how the operating system manages communication between applications and hardware.

3. Programming Language and Tools

Item	Description
Programming language	Python 3
Operating system	Parrot OS (Linux)
Editor	Kate
Storage interface	Linux `lsblk` command
Python modules	`os`, `shutil`, `subprocess`

4. Implementation

4.1 Device Selected

The implementation focuses on storage devices, such as internal hard drives and USB flash drives.

The program retrieves information about available storage devices and allows the user to enter a mounted storage path to check its capacity.

The implementation is read-only. It does not format, delete or modify data on storage devices.

4.2 System Interfaces and Functions Used

Function or command	Purpose
`subprocess.run()`	Executes the Linux `lsblk` command.
`lsblk`	Displays storage devices, their sizes, types and mount points.
`shutil.disk_usage()`	Retrieves total, used and available space for a filesystem.
`os.path.isdir()`	Checks whether the entered path is a valid directory.
`try` and `except`	Handle errors during device and filesystem operations.

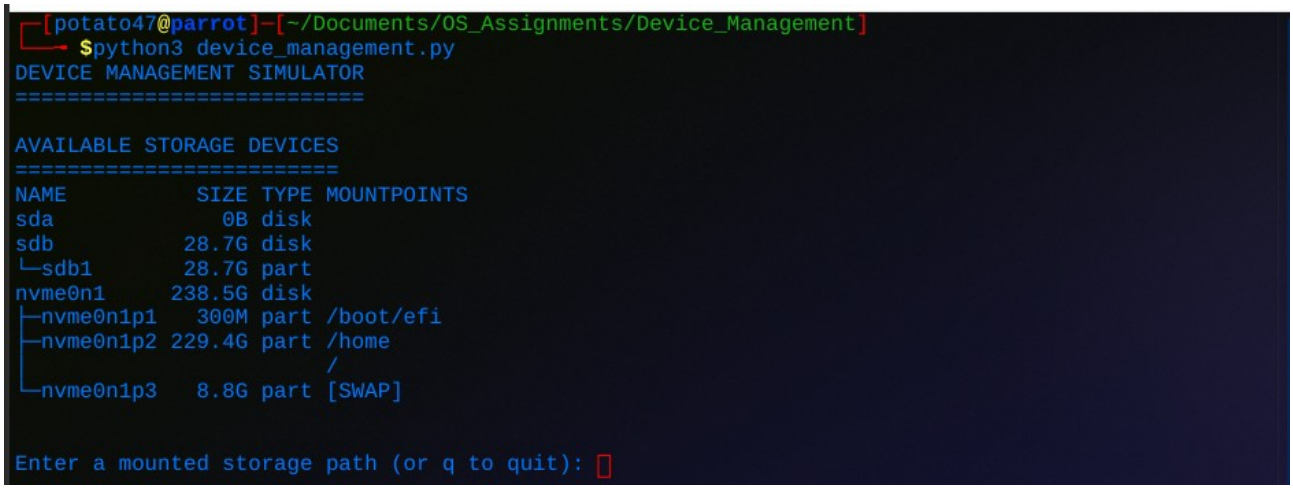
4.3 Program Operation

The program follows these steps:

1. Starts and displays the device management heading.
2. Executes `lsblk` to display available storage devices.
3. Requests a mounted storage path from the user.
4. Checks whether the entered path is valid.
5. Retrieves storage capacity information.
6. Displays total, used and available space.
7. Handles errors and allows the user to enter another path.

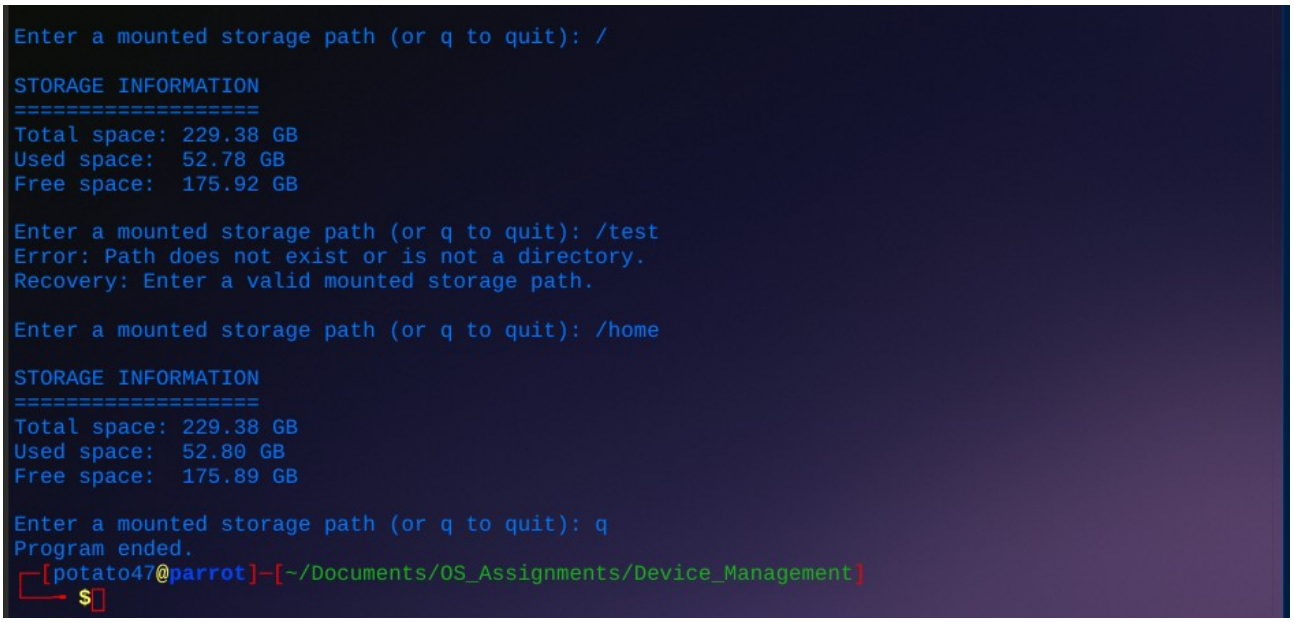
4.4 Compilation and Execution

Since the implementation uses Python, no separate compilation step is required.



5. Testing and Results

The program should be tested using different storage paths.



6. Error Handling and Recovery

Error handling prevents the program from terminating unexpectedly when a problem occurs.

The implementation handles the following situations:

- **Invalid path:** The program informs the user when the entered path does not exist or is not a directory and requests another path.
- **Permission error:** The program displays an error message when access to a storage path is denied.
- **Missing command:** If `lsblk` is unavailable, the program reports that the command could not be found.
- **Device or filesystem error:** The program displays the relevant operating system error and allows the user to try again.

These mechanisms make the program more reliable when interacting with storage devices.

7. How the Operating System Manages Device Communication

Applications do not normally communicate directly with hardware devices. Instead, they use interfaces provided by the operating system.

In this implementation:

1. The Python application requests information using system utilities and filesystem functions.
2. Linux provides access to storage-device information and filesystem statistics.
3. The kernel and device drivers manage communication with the storage hardware.

4. The retrieved information is returned to the application for display.

This demonstrates how the operating system acts as an intermediary between applications and hardware devices.

8. Discussion

The implementation demonstrates basic storage device management on Linux.

The use of `lsblk` allows the program to identify available storage devices and their mount points. The `shutil.disk_usage()` function retrieves capacity information for a selected mounted filesystem.

The program also demonstrates error handling and recovery by allowing users to correct invalid paths without restarting the application.

One limitation is that the program retrieves device and filesystem information rather than performing direct read and write operations on the hardware. However, it demonstrates how an application uses operating system interfaces to obtain information about storage devices safely.

9. Conclusion

The assignment demonstrates how a Python application can interact with storage devices through Linux system interfaces.

The program displays available storage devices, retrieves filesystem capacity information and handles common errors.

The implementation also illustrates the role of the operating system kernel and device drivers in managing communication between applications and hardware.

10. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
2. Linux Manual Pages. `lsblk(8)`. https://man7.org/linux/man-pages/man8/lsblk.8.html
3. Python Software Foundation. `shutil — High-level file operations`. https://docs.python.org/3/library/shutil.html
4. Python Software Foundation. `subprocess — Subprocess management`. https://docs.python.org/3/library/subprocess.html